

Observability

A hands-on course in building agents by building their harness

Lesson 7

The terminal scrolled past, and the answers went with it

Lessons 5 and 6 made the harness safe to leave running. Now you run it where you can't watch it:

- Why did that take four minutes?
- Why did it cost that much?
- What did it run, and in what order?
- Which command did the guard refuse, and which did the box kill?

A record of what the harness did, kept as it does it

- Every call to the model
- Every tool it ran
- How long each took, and how many tokens each cost
- Every time a guardrail said no

A harness you can't see into is one you can't trust or fix.

Same run, two records, two readers

	Episode (Lesson 4)	Trace (Lesson 7)
Written for	the model	you, the person running quark
Holds	what working memory would have been	timings, token counts, exit codes, refusals
Used to	search its own past	answer your questions afterwards

A record for the model is context. A record for the operator is observability.

Built on control flow

- The model interface only knows about one call
- Output only knows about one tool
- The loop is where a call, its tool, the guard's decision and the next call all happen, in order
- So the loop is the one place a record of the run can be made

No new primitive: a production layer adds hardening.

Read off what's already there, and write it down

- **A trace:** one JSON line per thing that happened, appended in order
- **Timing:** read the clock before and after a model call or a tool
- **Tokens:** every response says tokens in, out, and read from or written to the cache
- **A second opinion:** the exit code is rough, so keep Jev's "did it fail?" beside it

Observability only watches. It never changes what's sent, what runs, or when the loop stops.

The concept: `observability.py` (abridged)

```
def trace(**event):  
    with open("traces.jsonl", "a") as f: f.write(json.dumps({...}) + "\n")  
  
    for cmd in [...]:  
        start = time.time()  
        done = subprocess.run(cmd, shell=True, ...)  
        trace(event="tool", cmd=cmd, ..., exit=done.returncode, chars=len(done.stdout))  
  
    response = client.messages.create(model=..., messages=[...])  
    trace(event="model", seconds=..., input_tokens=..., output_tokens=...)
```

- `trace()` is the whole layer: append one JSON object per event
- Nothing is computed: the clock, the exit code and `usage` are read off

Run the concept: five lines in `traces.jsonl`

event	cmd	seconds	exit	chars
tool	<code>wc -l notes.txt</code>	0.0	0	12
tool	<code>grep -n TODO notes.txt</code>	0.0	1	0
tool	<code>ls missing.txt head -1</code>	0.0	0	59
tool	<code>sleep 2</code>	2.0	0	0
model	129 tokens in, 202 out	2.17		

The trace's fields as a table.

Ask the trace which commands failed

```
jq -c 'select(.event=="tool" and .exit != 0) | {cmd, exit}' traces.jsonl
```

```
{"cmd":"grep -n TODO notes.txt","exit":1}
```

- The `grep` found nothing: an answer, not a failure
- The `ls` that did fail isn't there: the pipe exited with `head`'s 0
- The model saw both; the trace only has the exit code, and it's rough

The concept with Jev: `jev_observability.py` (abridged)

`observability.py` plus Jev, and nothing else:

```
jev = TypeSafeClient(timeout=5) if os.environ.get("TYPESAFE_API_KEY") else None
FAILED = Noul(instructions="Does `result` show that the command failed ...?", ...)

def failed(cmd, result):
    try: return round(jev.system_one({...}, {"q": FAILED}).nouls["q"].noul, 2)
    except Exception: return None

    trace(event="tool", ..., failed=failed(cmd, done.stdout))
```

- `failed()` asks Jev one yes/no question, and gets a probability back
- It goes beside `exit`, never instead of it; no key or an error: `null`

Run it with Jev: the exit code and Jev disagree

cmd	exit	failed (Jev)
<code>wc -l notes.txt</code>	0	0.02
<code>grep -n TODO notes.txt</code>	1	0.06
<code>ls missing.txt head -1</code>	0	0.98
<code>sleep 2</code>	0	0.03

- One false alarm from the exit code, one real failure only Jev saw
- With a wrong key, every `failed` is `null`, and nothing else changes

quark: `trace()` and `failed()`, in control flow (abridged)

Lesson 6's `quark.py` plus 18 lines, 352 in all:

```
def trace(**event):  
    with open(".quark/traces.jsonl", "a") as f: f.write(json.dumps({...}) + "\n")  
def failed(cmd, result):  
    judged = ask({"command": cmd, "result": result[-4000:]}, FAILED)  
    return judged and round(judged["noul"], 2)
```

- Each line also carries `episode`: it ties the trace to the model's record of the same run
- `failed()` asks through Lesson 5's `ask()`: the last 4,000 characters, or `None`
- No `SURE` here: nothing is decided, so the probability is kept as it came

Then one call wherever something happens

```
trace(event="start", input=input)
    trace(event="stopped", steps=steps, tokens=spent)
    trace(event="too_long", drop=drop)
    trace(event="interrupted", during="saying")
        trace(event="refused", cmd=block.input["cmd"], why=no)
    trace(event="interrupted", during="acting")
```

- **start:** where a run begins and what came in
- **stopped:** the steps and tokens a limit stopped at
- **too_long:** the API said the prompt was too long, so compaction runs
- **refused:** the command the guard wouldn't run, and why
- **interrupted:** you pressed ESC, and whether quark was saying or doing something

The model and the tool lines (abridged)

```
start = time.time()
trace(event="model", seconds=..., stop_reason=..., input_tokens=..., ...)
```

```
start = time.time()
trace(event="tool", ..., failed=failed(block.input["cmd"], done.stdout))
```

- **model:** `stop_reason` and four counts from `usage`: input, output, cache read, cache write
- **tool:** the command, seconds, `exit`, characters printed, and Jev's `failed`
- The model never sees `failed`: its tool result is what the command printed

Run it: nobody there to answer the guard

```
quark.py "which file in this folder has the most lines? answer in one sentence" < /dev/null
```

```
$ wc -l * .[!.*]* 2>/dev/null | sort -n | tail -5
    0 .quark
    2 small.txt
 5000 big.txt
 5002 total
```

```
`big.txt` has the most lines, at 5,000.
```

Not on the safe list, but Jev was sure it only reads, so it ran without asking.

What you see afterwards, in `.quark/traces.jsonl`

event	seconds	stop / exit	in	out	cache_read	cache_write	failed
model	1.41	tool_use	95	76	0	8941	
tool	0.26	exit 0					0.08
model	0.99	end_turn	206	20	8941	0	

The trace's fields as a table; the `start` line is left out. Summed with `jq`, by episode:

```
{"calls":2,"input":301,"output":96,"cache_read":8941,"cache_write":8941,"seconds":2.4}
```

A run where things go wrong

A copy with `TIMEOUT` lowered to 5, and `y` piped in for the guard (shortened):

```
$ ls /nonexistent | head -1
ls: cannot access '/nonexistent': No such file or directory

$ grep TODO small.txt

$ sleep 60
allow `sleep 60`? (Jev: other, 0.81) [y/N]
(killed: ran over 5 seconds or out of memory)
```

The agent told you what happened, this time. Without the trace, you'd only know if you were watching.

Exit codes and Jev, side by side

```
jq -c 'select(.event=="tool") | {cmd, exit, failed}' .quark/traces.jsonl
```

```
{"cmd":"ls /nonexistent | head -1","exit":0,"failed":0.98}  
{"cmd":"grep TODO small.txt","exit":1,"failed":0.04}  
{"cmd":"sleep 60","exit":137,"failed":0.97}
```

- The same split as in the concept with Jev: Jev saw the hidden `ls`; the exit code flagged the `grep`
- They agree on the kill: `exit 137`
- Each line names its episode: from "what went wrong" to "what the model was thinking"

Press ESC, and the trace says why it stopped

In a terminal: `sleep 30`, `y` to the guard, ESC three seconds later (shortened):

```
{"event":"tool","cmd":"sleep 30","seconds":3.16,"exit":137,"chars":32,"failed":0.62}  
{"event":"interrupted","during":"acting"}
```

- Stopped at 3.16 seconds: `exit` 137, ESC's kill inside the box
- The next line says why: you interrupted it while it was acting
- Jev's 0.62: all it saw was `[your doing stopped before done]`
- The trace knows more about that command than the model or Jev does

Other things we could do

- **What's recorded:** events, *spans* with a parent (a tree), or every full request and response
- **Where it goes:** a file, the terminal, a database, an OpenTelemetry collector
- **When you see it:** afterwards, a live line per step, or a dashboard that alerts you
- **What's worked out:** tokens, cost, latency, cache hit rate, tool failures, steps per task
- **How you ask and how long it's kept:** `grep` and `jq`, SQL or a trace UI; sampling, rotation, deletion

A product on its own, and ideas worth knowing

Tracing platforms like Langfuse and LangSmith are this layer: you send them spans, they store them, price them and show them as a tree.

- **Spans with parents:** the run is the parent of every call and tool; a crash is recorded and still crashes
- **Jev in its own span**, a child of the tool's, so the tool's time stays the tool's
- **Cost:** token counts times your provider's rates, kept on each model call
- **A live line** per step, **a summary** per run, **a report** across every run

The rule

Record what the harness does as it does it: each model call, each tool, each refusal, how long it took and what it cost, in a log you can search afterwards.

Do it where control flow already sees the whole sequence, read from what's already there, and keep it apart from what the model remembers. Keep a second opinion beside a rough signal, not instead of it.

What observability never does

- **Control flow:** it sits in the loop, but never decides what runs next or when to stop
- **Model interface:** sends and receives as before; Jev's answer goes only into the trace
- **Input:** it gathers none
- **Context:** the trace is not in the request; that's what makes it observability
- **Output:** runs tools the way it always did, only with a clock around them

What's missing: it watches, and that's all

It will faithfully record that:

- the API dropped a call halfway through a long run
- the model was cut off in the middle of a command
- you pressed ESC, and everything it had said or printed up to then was thrown away
- the process died and took the work with it

It can tell you exactly where things broke; it can't pick up from there. A harness that runs unattended has to survive a bad day, not just describe one.

Next: Resilience

Lesson 8